

자료구조 및 알고리즘

Problem 01

본문에서 파이썬 내장 리스트를 사용하는 큐를 구현할 때 리스트의 맨 앞을 큐의 front로 리스트의 맨 끝을 큐의 tail로 간주했다. 반대로 리스트의 맨 끝을 큐의 front로 리스트의 맨 앞을 큐의 tail로 간주하여 다음 listQueue.py 코드를 바꾸어 보시오.

```
class ListQueue:
    def __init__(self):
        self.__queue = []

    def enqueue(self, x):
        self.__queue.append(x)

    def dequeue(self):
        return self.__queue.pop()

    def front(self):
        if self.isEmpty():
            return None
        else:
            return self.__queue[0]
```

✓
def isEmpty(self)->bool:
return len(self.__queue) == 0

✓
def dequeueAll(self):
self.__queue.clear()

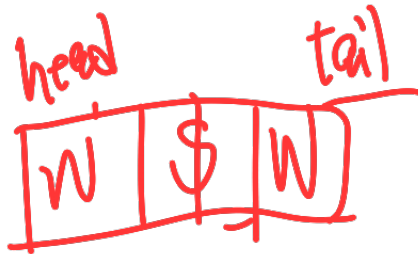
✓
def printQueue(self):
print("Queue from front:", end=' ')
for i in range(len(self.__queue)):
 print(self.__queue[i], end=' ')
print()

☆ 출제예상.

Problem 02

입력으로 들어온 문자열이 다음 집합의 원소인지 체크하는 코드를 큐를 이용해서 작성하시오.

ab\$ab 이런거가..?



{w\$w | w: 문자열}

```
def checkStr(s):
    q = ListQueue()
    for i in range(len(s)):
        if s[i] == '$': break
        q.enqueue(s[i])
    for j in range(i+1, len(s)):
        if q.isEmpty(): return False
        if s[j] != q.dequeue(): return False
    if q.isEmpty(): return True
    else: return False
```

큐 a → 큐 c에 옮긴다.

큐 c에서 → 큐 a와 큐 b에 넣는다.

Problem 03

LinkedList 타입의 객체 a의 내용을 그대로 또 다른 LinkedList 타입의 객체 b로 복사하는 코드를 작성하시오. (a의 레퍼런스 값을 b로 복사하여 두 변수가 같은 레퍼런스를 가져 내용이 같아지도록 하라는 것이 아니라 내용 복사를 요구하는 것이다.)

```
def copyQueue(a,b):
```

```
    c=LinkedList()
```

```
    while(!a.isEmpty()):
```

```
        c.enqueue(a.dequeue())
```

```
    b.dequeueAll()
```

```
    while(!c.isEmpty()):
```

```
        temp=c.dequeue()
```

```
        a.enqueue(temp)
```

```
        b.enqueue(temp)
```

Problem 04

비효율적이지만 2개의 큐를 사용하여 스택의 push()와 pop() 알고리즘을 작성하시오.

Q1은 스택 역할 ← 1 1 2 1 3 → 둘다. | 1 |
Q2는 temp 역할 ← 4 1 1 2 1 3
Q1에 다시대입.

큐에 스택 순서로 정리해서 넣겠다는 것.

```
class QueueStack:
    def __init__(self):
        q1 = ListQueue()
        q2 = ListQueue()
    def push(self, x):
        self.q2.enqueue(x)
        #그다음 Q1에 있는 것 Q2로 이동
        while(not q1.IsEmpty()):
            q2.enqueue(q1.dequeue())
        self.q1, self.q2 = self.q2, self.q1
    def pop(self):
        return self.q1.dequeue()
```

Problem 05

비효율적이지만 2개의 스택을 사용하여 큐의 enqueue()와 dequeue() 알고리즘을 작성하시오.

```
class StackQueue:
```

```
    def __init__:
```

```
        S1 = ListStack()
```

```
        S2 = ListStack()
```

```
    def enqueue(self, x):
```

```
        self.S1.push(x)
```

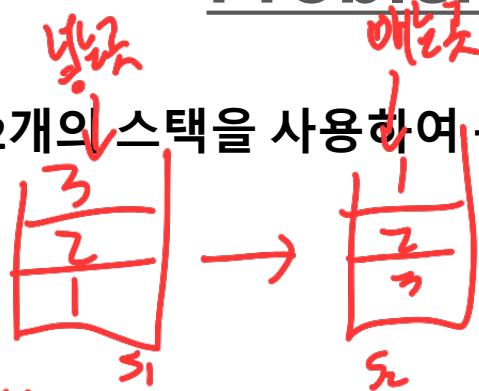
```
    def dequeue(self, x):
```

```
        if (S2.IsEmpty()):
```

```
            while(not S1.IsEmpty()):
```

```
                S2.push(S1.pop())
```

```
        return S2.pop()
```



S2 비어있을 때만 S1 pop 해서 S2에 넣어.

Problem 06



이 장에서 배운 큐에서 enqueue()는 항상 큐의 맨 뒤에, dequeue()는 항상 큐의 맨 앞에서 이루어진다. Deque는 이의 변형으로 enqueue()와 dequeue()가 큐의 맨 앞과 맨 뒤에서 다 가능하다. [코드 7-13]의 클래스 환경에서 이를 가장 효율적으로 구현한다고 하자. 큐의 사이즈가 n 일 때, enqueue()와 dequeue()의 수행 시간은 각각 어떻게 되겠는가?

① enqueue 앞,
 $O(1)$

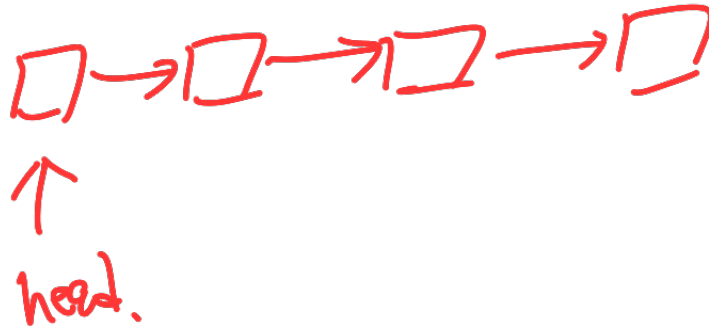
③ dequeue 앞,
 $O(1)$

② enqueue 뒤,
 $O(1)$

④ dequeue 뒤,
 $O(n)$

Problem 07

06번 문제에서 deque를 원형 연결 리스트가 아닌 [코드 5-15]의 LinkedListBasic 객체를 사용한다면 enqueue()와 dequeue()의 수행 시간은 각각 어떻게 되겠는가



① enqueue() 앞
 $O(1)$

② enqueue() 뒤
 $O(n)$

③ dequeue()
 $O(1)$

④ dequeue()
 $O(n)$

Problem 08

다음은 클래스 ListQueue의 파이썬 코드다. 이를 06번 문제의 자료구조 Deque를 구현하는 클래스 Deque로 바꾸어 보시오. 필요한 메서드를 추가하시오.

앞뒤여부 추가됨

```
class ListQueue:
    def __init__(self):
        self.__queue = []

    def enqueue(self, x):
        self.__queue.append(x)

    def dequeue(self):
        return self.__queue.pop(0)

    def front(self):
        if self.isEmpty():
            return None
        else:
            return self.__queue[0]
```

```
def isEmpty(self)->bool:
    return len(self.__queue) == 0

def dequeueAll(self):
    self.__queue.clear()

def printQueue(self):
    print("Queue from front:", end=' ')
    for i in range(len(self.__queue)):
        print(self.__queue[i], end=' ')
    print()
```

```
def enqueueTail(self, x):  
    self.__queue.append(x)
```

```
def enqueueFront(self, x):  
    self.__queue.insert(0, x)
```

```
def dequeueTail(self, x):  
    self.__queue.pop()
```

```
def dequeueFront(self, x):  
    self.__queue.pop(0)
```